

个人报告：STM32云台控制子系统设计与实现

姓名：章涵 选题：题目4 · 昇腾310B目标跟踪检测 主要负责：STM32云台控制子系统开发、硬件联调 提交日期：2026-07-13

一、个人工作概述

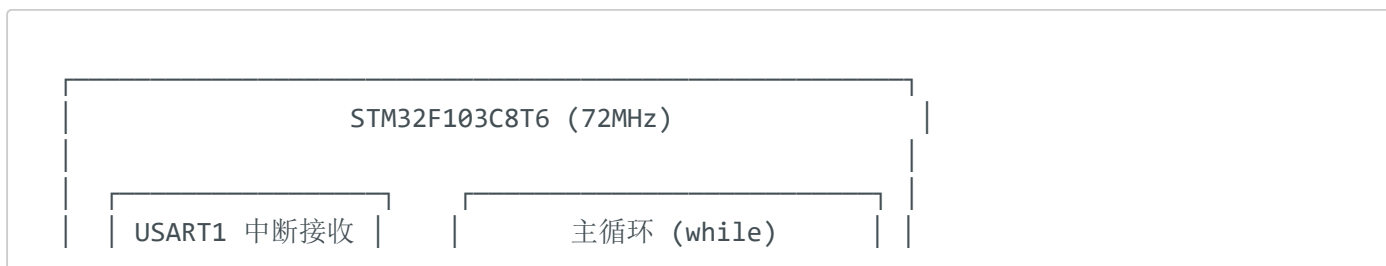
在本项目中，我独立负责STM32F103C8T6云台控制子系统的完整开发，从零编写了全部固件代码，具体包括：

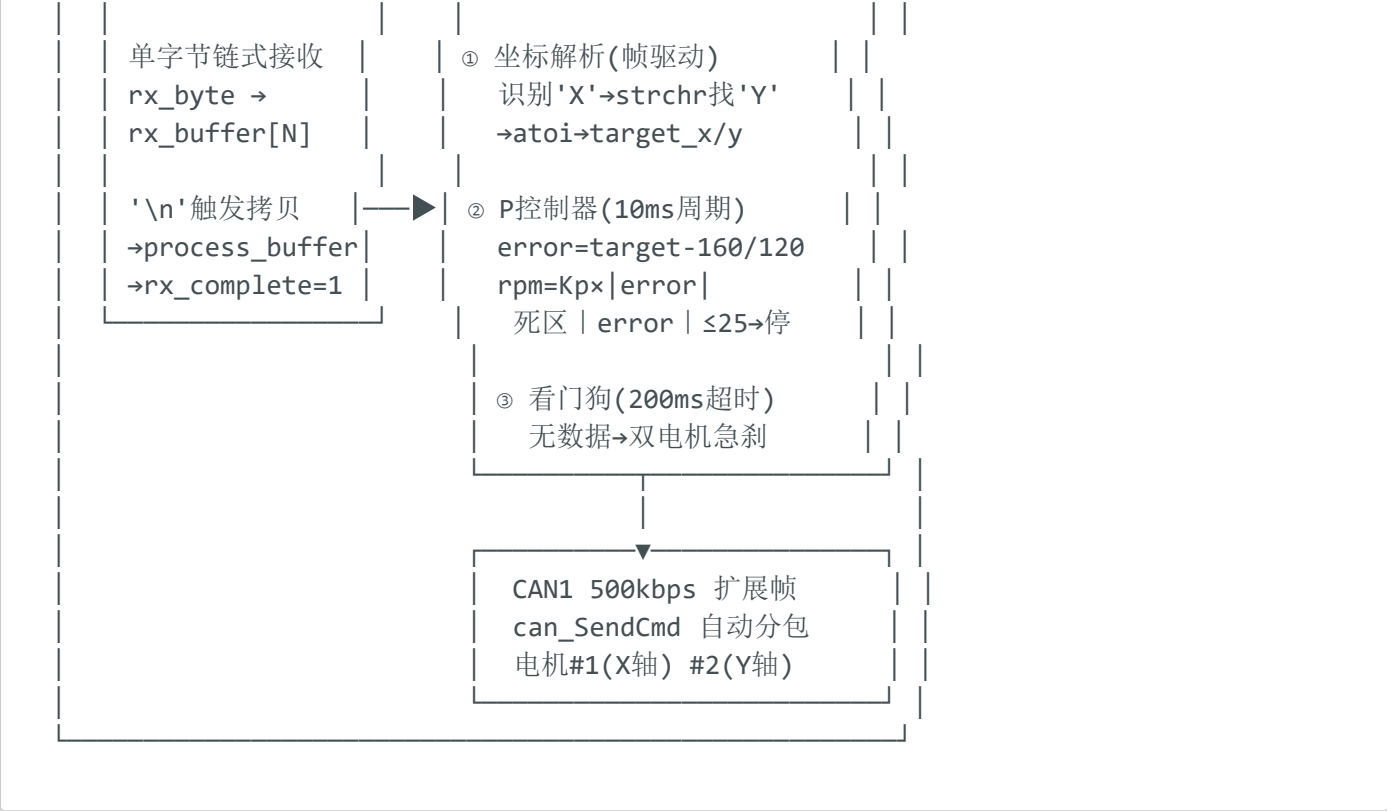
- STM32CubeMX工程搭建与CMake构建配置
- UART串口中断接收（单字节链式中断 + 双缓冲机制）
- 视觉坐标协议解析（"X{nx}Y{ny}\n"格式）
- P控制器设计与实现（速度环 + 死区 + 限速）
- CAN总线驱动开发（500kbps扩展帧 + 自动分包算法）
- Emm_V5.0闭环步进电机协议封装（50+个API函数）
- 看门狗安全机制（200ms无数据自动急刹）
- CAN应答中断风暴问题的发现与解决
- 硬件连接与全链路联调

我的核心任务是：让STM32听懂昇腾发来的坐标，算出差多少，然后精准地驱动电机追上去——而且要在目标丢了的时候安全停下来。

二、系统架构设计

2.1 STM32端总体架构





2.2 为什么选择速度环

这是我在设计控制策略时做出的核心决策。两种方案的对比如下：

维度	速度环（采用）	位置环
控制方式	偏差→转速，越偏越快	偏差→目标角度，转到即停
目标持续移动	✅ 始终平滑追赶	❌ "到达-停-发现偏了-再追"锯齿运动
目标在中心附近	死区内停转，画面静止	到达后停转，画面也静止
响应速度	偏差大时转速高，快	固定速度转到目标角度
画面效果	平滑连续	一顿一顿

结论：实时追踪场景不像机械臂定位——目标不会等你到了再动。速度环让电机保持追赶姿态，配合死区防止中心区域振荡，是更合适的选择。

三、UART串口通信设计

3.1 单字节中断链式接收

串口接收是我遇到的设计难点之一。昇腾端发来的数据是变长的（X1Y2\n到X320Y240\n），不能按固定长度接收。

我采用了**单字节中断链式接收方案**：

```
// 初始化：启动接收一个字节
HAL_UART_Receive_IT(&huart1, &rx_byte, 1);

// 回调函数：每收到一个字节自动触发
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    if (rx_byte == '\n') {
        // 帧结束：拷贝到process_buffer，设标志位
        rx_buffer[rx_index] = '\0';
        strcpy(process_buffer, rx_buffer);
        rx_complete = 1;
        rx_index = 0;
    } else if (rx_byte != '\r') {
        // 忽略回车，追加到缓冲区
        if (rx_index < RX_BUF_SIZE - 1)
            rx_buffer[rx_index++] = rx_byte;
    }
    // 关键：重新挂载接收中断，形成链式循环
    HAL_UART_Receive_IT(&huart1, &rx_byte, 1);
}
```

设计要点：

- 每次只收1字节，中断延迟极低
- 回调末尾立即重新挂载（链式），保证永远在收
- \n为帧结束标志，与昇腾端的发送格式匹配
- 主动过滤\r（回车符，部分串口工具会插入）

3.2 双缓冲防竞争

这是一个重要的并发安全设计：

```
rx_buffer      → ISR中写入（中断上下文）
process_buffer → 主循环中读取（线程上下文）

帧结束时：strcpy(process_buffer, rx_buffer) → rx_complete = 1
```

为什么需要双缓冲？

如果只有一个缓冲区，主循环正在atoi解析到一半时，新的中断到来覆盖了数据——解析结果就会出错。双缓冲确保ISR和主循环各操作各自的缓冲区，只在帧结束的瞬间

strcpy拷贝。

为什么strcpy是安全的？

strcpy在ARM Cortex-M3上编译为strb指令序列，操作在微秒级完成。而下一帧数据至少间隔50ms（昇腾端限频20Hz），不存在连续打断的风险。

3.3 协议解析

昇腾端发送的格式是"X{nx}Y{ny}\n"，例如"X160Y120\n"表示目标在画面正中心。

解析算法：

```
if (process_buffer[0] == 'X') {
    char *y_ptr = strchr(process_buffer, 'Y'); // 找到'Y'的位置
    if (y_ptr != NULL) {
        *y_ptr = '\0'; // 在'Y'处临时截断
        target_x = atoi(&process_buffer[1]); // "160"
        target_y = atoi(y_ptr + 1); // "120"
        target_valid = 1;
        last_rx_time = HAL_GetTick(); // 喂狗
    }
}
```

用标准库函数strchr和atoi而不是手写解析，代码简洁可靠。

四、P控制器设计

4.1 核心算法

```
error_x = target_x - 160; // CENTER_X = 320/2
error_y = target_y - 120; // CENTER_Y = 240/2

if (abs(error_x) > 25) { // 死区：25像素
    rpm_x = abs(error_x) * 0.1; // 比例控制：Kp=0.1
    rpm_x = clamp(rpm_x, 0, 150); // X轴限速150RPM
    dir_x = (error_x > 0) ? 1 : 0; // 正向/反向
}
// 死区内：rpm=0，不发送命令 → 电机停转

Emm_V5_Vel_Control(1, dir_x, rpm_x, ACCEL, 0); // CAN发X轴
```

```
HAL_Delay(1); // 错峰1ms
Emm_V5_Vel_Control(2, dir_y, rpm_y, ACCEL, 0); // CAN发Y轴
```

4.2 参数设计思路

参数	值	设计理由
Kp_vel_x	0.1	最大偏差160px→16RPM，实际大多在50~100RPM。初始设0.2，但出现过冲振荡，降到0.1
Kp_vel_y	0.05	Y轴（俯仰）重力分量会使电机下坠，更低增益防止抬头低头过快
DEADZONE	25px	占画面约8%。初始设15，发现目标在中心附近时电机来回蹭，加大到25后画面稳定
MAX_RPM_X	150	水平方向无重力干扰，可以快一些
MAX_RPM_Y	60	俯仰方向限速，防止重力导致云台猛然低头
ACCEL	15	平滑启停，避免突然启动的冲击力
控制周期	10ms	100Hz足够，20Hz的串口输入和20ms的HAL_Delay误差可以接受

4.3 为什么不用PID而用纯P

有人可能会问为什么不加积分项（I）和微分项（D）。我的考虑是：

- **I（积分）**：会累积历史偏差。目标移动时偏差不会收敛到零（因为速度环持续追赶），积分会一直累积然后超调再回调，反而引入振荡。跟踪场景不需要"零稳态误差"——目标在中心±25px就够了。
- **D（微分）**：对噪声敏感。检测框本身有抖动，EMA平滑后的坐标虽然稳了，但微分项仍然可能被高频噪声放大。

纯P控制器在这个场景下足够：误差大→转得快→误差减小→转速减小→进入死区→停止。简单、稳定、可预测。

4.4 错峰发送

两个电机命令之间加HAL_Delay(1)不是随意为之：

原因：STM32F103的bxCAN控制器只有**3个发送邮箱**。连续发两帧数据填满邮箱后，第三帧（如果存在）会返回**HAL_CAN_BUSY**。1ms间隔足够CAN控制器把上一帧从邮箱发送到总线上，清出一个空位。

五、CAN总线驱动开发

5.1 波特率配置

CAN波特率计算： **$500\text{kbps} = 72\text{MHz} / 12(\text{Prescaler}) / (1+4+1)$**

```
SYSCLK=72MHz → APB1=36MHz → CAN时钟=36MHz
Prescaler = 12 → CAN时钟 = 36/12 = 3MHz
Time Quanta = 1/(3MHz) ≈ 333ns
Bit = (1+4+1)×333ns = 2μs
波特率 = 1/(2μs) = 500kbps
```

500kbps是Emm_V5驱动器的默认速率，也是工业CAN总线最常用的速率。

5.2 扩展帧ID编码

```
ExtId = (电机地址 << 8) | 分包序号
        └─ 高8位 ─┘   └─ 低8位 ─┘
```

```
例：电机#1的第0包 → ExtId = (1 << 8) | 0 = 0x100
     电机#2的第3包 → ExtId = (2 << 8) | 3 = 0x203
```

这样编码后，接收方可以根据ExtId直接判断是哪个电机的哪一分包，无需额外协议头。

5.3 自动分包算法

CAN标准帧每帧最多8字节数据。Emm_V5的速度控制命令恰好8字节（含功能码1字节+数据6字节+校验1字节=8字节），不需要分包。但位置控制命令13字节，超出单帧容量。

我在**can_SendCmd**中实现了通用的自动分包：

cmd[0]=地址, cmd[1]=功能码, cmd[2..len-2]=数据, cmd[len-1]=0x6B(校验)

每子帧: [功能码][数据0..数据6] → DLC=1+min(7, 剩余数据)
packNum从0递增

六、CAN应答中断风暴——最难忘的Bug

6.1 故障现象

这是整个项目中让我印象最深的问题。初始化阶段开启CAN RX中断后(HAL_CAN_ActivateNotification)，系统直接卡死在初始化阶段。不是偶尔卡，是每次必卡。用调试器看，程序停在HAL_Delay(500)里永远出不来。

6.2 原因分析

我花了几乎一整个下午定位这个问题。根因链路如下：

- ① 第一个电机命令（HAL_Delay(500)前的初始化序列）发出
- ② 电机收到后立即回一帧应答
- ③ CAN RX中断触发 → 进入ISR → 读取FIFO
- ④ 在ISR中读取应答时，电机可能又发了新应答（如状态上报）
- ⑤ 中断嵌套或连续触发 → 中断风暴
- ⑥ CAN RX中断优先级(比SysTick高) → SysTick永远得不到响应
- ⑦ HAL_GetTick()值不更新 → HAL_Delay()死循环

关键点：Emm_V5驱动器在接收每条有效命令后都回一帧应答（这是驱动器协议的设计）。如果CAN RX中断开着，每个应答都会触发中断。CAN中断的NVIC优先级默认高于SysTick，导致HAL_Delay的计时基准uwTick无法更新。

6.3 解决方案

```
// 不开接收中断：本程序只发不收
// 电机每条命令都会回一帧应答，若开RX中断又不及时读FIFO，
// 应答帧会触发中断风暴(优先级高于SysTick)，卡死 HAL_Delay
// if (HAL_CAN_ActivateNotification(&hcan, CAN_IT_RX_FIFO0_MSG_PENDING) != HAL_OK)
//     { Error_Handler(); }
```

最终方案：注释掉RX中断注册，保留CAN RX中断NVIC配置（避免HAL库报配置错误），但实际不触发回调。本项目场景中STM32确实不需要接收电机应答——只管发速度命令。

这个Bug教会我：**中断优先级的分配不是随意的，必须考虑外设之间的时序依赖。**
HAL_Delay依赖SysTick→SysTick优先级不能低于任何会频繁触发的中断。

七、看门狗安全机制

7.1 设计思路

在云台自动追踪中，有一个重要的安全场景：**目标消失在画面外**（比如人走出了摄像头视野）。

这时昇腾端停止发送坐标，STM32端如果继续执行最后一次收到的坐标对应的速度，电机可能会一直转下去——可能转到机械极限位置卡死甚至损坏。

7.2 实现

```
// 每次收到有效坐标时
last_rx_time = HAL_GetTick();
motors_stopped = 0;

// 主循环每次迭代检查
if (current_time - last_rx_time > 200 && motors_stopped == 0) {
    Emm_V5_Vel_Control(1, 0, 0, ACCEL, 0); // 电机1急刹
    HAL_Delay(1);
    Emm_V5_Vel_Control(2, 0, 0, ACCEL, 0); // 电机2急刹
    motors_stopped = 1; // 防止重复发送急刹命令
}
```

200ms超时的设计依据：昇腾端限频20Hz发送坐标，即每50ms发一帧。200ms $\approx$ 4帧间隔，足够容忍偶尔的丢帧或通信延迟，又能在真正目标丢失后及时停下。

motors_stopped标志位防止在已经停止后重复发送急刹命令，减少不必要的CAN总线流量。

八、Emm_V5电机协议封装

我封装了张大头Emm_V5.0闭环步进伺服驱动器的完整CAN通信协议，共**50+个API函数**，涵盖：

功能类别	API数量	主要函数
基本控制	3	En_Control（使能），Vel_Control（速度模式），Pos_Control（位置模式）
急停与同步	2	Stop_Now, Synchronous_motion
原点回归	4	Origin_Set_0, Origin_Trigger_Return, Origin_Interrupt
参数读写	15+	修改电机ID、微步细分、PID参数、电流环参数、方向等
状态读取	14	读取总线电压、电流、位置、速度、温度、标志位等
多命令链	全套	MMCL_*系列函数，支持多命令排队一次执行

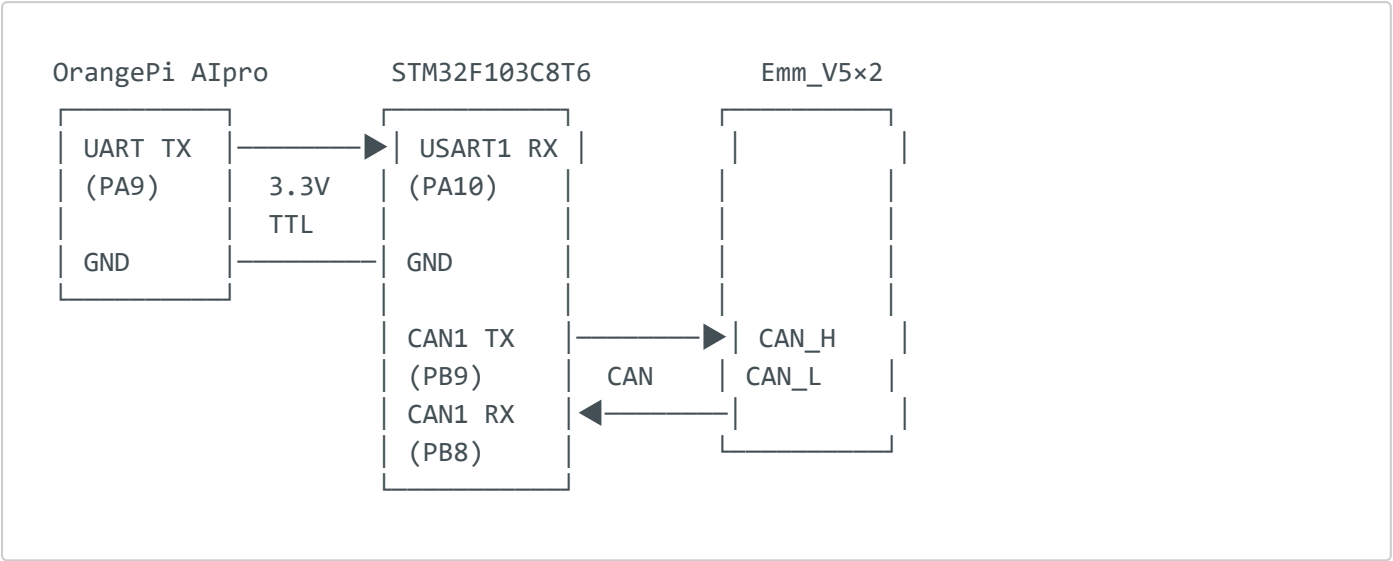
核心API——速度模式（主用）：

```
void Emm_V5_Vel_Control(uint8_t addr, uint8_t dir, uint16_t vel, uint8_t acc, bool snF)
{
    cmd[0] = addr;           // 电机地址 #1或#2
    cmd[1] = 0xF6;           // 功能码：速度模式
    cmd[2] = dir;            // 0=CW, 1=CCW
    cmd[3] = vel >> 8;       // 转速高字节 (0-5000 RPM)
    cmd[4] = vel & 0xFF;     // 转速低字节
    cmd[5] = acc;            // 加速度 (0-255)
    cmd[6] = snF;            // 同步标志
    cmd[7] = 0x6B;           // 固定校验字
    can_SendCmd(cmd, 8);
}
```

每帧8字节CAN数据，格式对应驱动器的**0xF6**速度控制指令。

九、硬件连接与联调

9.1 硬件连接方案



9.2 联调步骤

1. **STM32单独测试**：用电脑串口助手发X100Y120\n，确认电机正确响应
2. **昇腾单独测试**：运行tracking_app.py，在终端观察串口输出的坐标日志
3. **两板对接联调**：OrangePi UART → STM32 USART1，全链路运行
4. **负载测试**：安装摄像头到云台支架上，真实场景跟踪测试

9.3 Y轴方向校正

首次安装云台后发现目标向上移时电机反而向下转——Y轴方向装反了。由于代码中已预留方向切换逻辑：

```
dir_y = (error_y > 0) ? 1 : 0; // 方向反了就把 1:0 对调
```

只需把1:0对调为0:1即可修正，不需要改硬件接线。

十、个人收获与反思

10.1 技术收获

1. **从零搭建STM32工程的完整流程**：STM32CubeMX配置→CMake构建→ARM GCC编译→烧录→调试。这不是简单的"跑例程"，而是从空工程搭建到全部固件自主编写。
2. **实时控制系统的设计思维**：速度环vs位置环的选择、死区的意义、限速的必要性——这些不是书上写的，而是在实际调试中"撞了墙"才真正理解的。速度环+死区的组合是反复调参试出来的最优方案。
3. **中断优先级与实时性**：CAN中断风暴是项目中最棘手的问题，也是最有价值的教训。它让我深刻理解了嵌入式系统中"中断优先级决定系统生死"——一个看似无害的中断，因为优先级高于SysTick就可能导致整个系统假死。
4. **CAN总线通信**：CAN的差分信号、扩展帧格式、波特率计算、自动分包——从完全不懂到能写驱动，这是本项目的核心预习成果。CAN协议的可靠性远高于UART，适合工业场景。
5. **跨处理器协作**：昇腾和STM32是两个独立运行的系统，通过最简单的"发坐标→收坐标"连接。但要让两个系统配合好，需要统一坐标系（归一化）、约定通信协议（ASCII + \n）、设计容错机制（看门狗 + 限频）。

10.2 可以改进的地方

1. **控制算法升级**：纯P控制器虽然简单稳定，但在目标快速变速时响应不够灵敏。可以尝试**前馈+反馈**：用卡尔曼滤波器预测的速度作为前馈量，让电机"预判"目标的下一步位置。
2. **双缓冲可以更优雅**：当前**strcpy**拷贝数据，可以用环形缓冲区（ring buffer）实现零拷贝，但在这个数据量下收益不大。
3. **电机状态监控**：当前因CAN中断风暴问题关闭了接收，无法读取电机状态（电流、温度、位置）。如需监控，可以使用间隔查询而不是中断方式。

10.3 项目感悟

这个项目让我真正体会到了"软硬件协同"的意义。昇腾是大脑——它想清楚目标在哪；STM32是小脑——它负责把"想"变成"动"。而连接它们的，就是我设计的那条串口线和通信协议。

最让我自豪的不是写出了多复杂的代码，而是在遇到**CAN中断风暴**这种"系统直接卡死"的问题时，没有绕过去、换方案，而是一步步分析根因、最终找到解决方案。这种解决问题的能力，比任何一行代码都更有价值。

另外，Code Review也是重要的能力。Emm_V5的封装代码有1000+行，我花了大量时间阅读驱动器的开发手册、理解每个功能码的含义，才能写出准确的CAN帧。别人提供的SDK不是拿来直接用的——要看懂、要适配、要确保每帧的校验字都对。

十一、总结

我在本项目中的核心贡献是：**从零独立完成了STM32云台控制子系统的全部固件开发，打通了"串口接收→坐标解析→P控制→CAN发送→电机驱动→安全保护"的完整链路。**

从硬件连接到协议设计，从控制算法到中断调试，从参数调优到安全机制，我独立面对并解决了每一个环节的问题。最终实现了摄像头云台平滑随动——目标走到哪，云台跟到哪。这不是看代码看出来的，是一行行写、一次次烧录、一遍遍调试跑出来的。

个人报告结束